

Day 50 - Hibernate Day 08

One-to-One Mapping Notes

One-to-One Mapping connects one record of one table with only one record of another table. Example: One Question has one Answer and one Answer belongs to one Question.

Question Entity

```
@Entity
@Table(name = "que_value")
@Data
public class Question {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "que_id")
    private int id;

    private String quetions;

    @OneToOne(mappedBy = "question")
    @JoinColumn(name = "ans_id")
    private Answers answers;
}
```

Question is mapped to the que_value table and stores one related Answer object.

Answers Entity

```
@Entity
@Table(name = "ans_value")
@Data
public class Answers {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "ans_id")
    private int id;

    private String answers;

    @OneToOne
    private Question question;
}
```

Each Answer belongs to exactly one Question.

Main Program (Relationship Part)

```
Session currentSession =
    ConnectionEstablisher.getConnection().openSession();

Transaction transaction =
    currentSession.beginTransaction();

Question que1 = new Question();
Question que2 = new Question();
Question que3 = new Question();

Answers ans1 = new Answers();
Answers ans2 = new Answers();
Answers ans3 = new Answers();

que1.setAnswers(ans2);
que2.setAnswers(ans1);
que3.setAnswers(ans3);

ans1.setQuestion(que2);
```

```

ans2.setQuestion(que1);
ans3.setQuestion(que3);

currentSession.persist(que1);
currentSession.persist(que2);
currentSession.persist(que3);

currentSession.persist(ans1);
currentSession.persist(ans2);
currentSession.persist(ans3);

Question reference =
    currentSession.getReference(Question.class, 2);

System.out.println(reference.getQuestions());
System.out.println(reference.getAnswers().getAnswers());

transaction.commit();
currentSession.close();

```

Understanding @OneToOne(mappedBy = "question")

Both classes know each other, so Hibernate needs to know which class manages the relationship.

mappedBy tells Hibernate that the relationship is already managed by the question field inside the Answers class.

```

@OneToOne
private Question question;

```

Because the field name is question, we write mappedBy = "question".

Owner Side: Answers manages the relationship.

Inverse Side: Question only follows it.

If mappedBy is not written: Hibernate may treat both classes as owners, creating duplicate relationship mappings and extra foreign keys.

@JoinColumn

@JoinColumn specifies the foreign key column name.

```

@OneToOne
@JoinColumn(name = "question_id")
private Question question;

```

Without @JoinColumn, Hibernate automatically creates a default foreign key column name.

Inserted Data

Question	Answer
What is <mapping> work?	It is use map the object to the database
What is hibernate?	Hibernate is an ORM framework
What is use of @DependsOn ?	execute the dependent class for one class

Fetch Output

```

What is hibernate?
Hibernate is an ORM framework

```

Database Tables

Table: que_value

QUE_ID	QUESTIONS
1	What is <mapping> work?
2	What is hibernate?
3	What is use of @DependsOn ?

Table: ans_value

ANS_ID	ANSWERS	QUESTION_QUE_ID
1	Hibernate is an ORM framework	2
2	It is use map the object to the database	1
3	execute the dependent class for one class	3

Execution Flow

- Open Hibernate Session.
- Begin Transaction.
- Create Question objects.
- Create Answer objects.
- Connect both objects using setters.
- Persist Question and Answer objects.
- Fetch Question using getReference().
- Access the related Answer through the One-to-One relationship.
- Commit the transaction.
- Close the Session.

Quick Revision

Annotation	Purpose
@OneToOne	Creates a One-to-One relationship
mappedBy	Marks the inverse side
@JoinColumn	Creates or renames the foreign key column